

UT.6.4 PROGRAMACIÓN EN BASES DE DATOS: EXCEPCIONES

1. INTRODUCCIÓN	2
2. MANEJADORES DE EXCEPCIONES	2
3. EXCEPCIONES DEFINIDAS POR EL USUARIO	5
4. MENSAJES DE ERROR DE USUARIO	6
5. PROPAGACIÓN DE EXCEPCIONES	7

1.INTRODUCCIÓN

Las **excepciones** son unas características de PL/SQL cuyo objetivo es el tratamiento de los **errores** que se producen en **tiempo de ejecución** y no de compilación.

Cuando se produce un error, se genera una excepción. Cuando esto sucede, el control pasa al gestor de excepciones, que es una sección independiente del bloque. Una vez aquí no hay forma de volver a la sección ejecutable del bloque.

Las excepciones pueden ser:

- **Implícitas**: que son predefinidas y no predefinidas por el servidor Oracle
- **Explicitas**: que son las definidas por el usuario.

2.MANEJADORES DE EXCEPCIONES

Cualquier situación de **error** es llamada **excepción** en PL/SQL. Cuando se detecta un error, una **excepción es lanzada**, es decir, la ejecución normal se para y el control se transfiere a la parte de **manejo de excepciones**. La parte de manejo de excepciones es la parte etiquetada como **EXCEPTION** y consta de sentencias para el manejo de dichas excepciones, llamadas manejadores de excepciones. Sintaxis:

```
WHEN nombre_excepcion THEN
<sentencias para su manejo>
....
WHEN OTHERS THEN
<sentencias para su manejo>
```

La parte OTHERS captura cualquier excepción no considerada. Las excepciones pueden estar definidas por el usuario o definidas internamente. Las excepciones predefinidas se lanzarán automáticamente asociadas a un error de Oracle. Son las siguientes:

Nombre	Descripción	Código Oracle
ACCESS_INTO_NULL	El programa intentó asignar valores a los atributos de un objeto no inicializado	ORA-6530
COLLECTION_IS_NULL	El programa intentó asignar valores a una tabla anidada aún no inicializada	ORA-6531
CURSOR_ALREADY_OPEN	El programa intentó abrir un cursor que ya se encontraba abierto. Recuerde que un cursor de ciclo FOR automáticamente lo abre y ello no se debe especificar con la sentencia OPEN	ORA-6511

DUP_VAL_ON_INDEX	El programa intentó almacenar valores duplicados en una columna que se mantiene con restricción de integridad de un índice único (unique index)	ORA-1
INVALID_CURSOR	El programa intentó efectuar una operación no válida sobre un cursor	ORA-1001
INVALID_NUMBER	En una sentencia SQL, la conversión de una cadena de caracteres hacia un número falla cuando esa cadena no representa un número válido	ORA-1722
LOGIN_DENIED	El programa intentó conectarse a Oracle con un nombre de usuario o password inválido	ORA-1017
NO_DATA_FOUND	Una sentencia SELECT INTO no devolvió valores o el programa referenció un elemento no inicializado en una tabla indexada	ORA-100
NOT_LOGGED_ON	El programa efectuó una llamada a Oracle sin estar conectado	ORA-1012
PROGRAM_ERROR	PL/SQL tiene un problema interno	ORA-6501
ROWTYPE_MISMATCH	Los elementos de una asignación (el valor a asignar y la variable que lo contendrá) tienen tipos incompatibles. También se presenta este error cuando un parámetro pasado a un subprograma no es del tipo esperado	ORA-6504
SELF_IS_NULL	El parámetro SELF (el primero que es pasado a un método MEMBER) es nulo	ORA-30625
STORAGE_ERROR	La memoria se terminó o está corrupta	ORA-6500
SUBSCRIPT_BEYOND_COUNT	El programa está tratando de referenciar un elemento de un arreglo indexado que se encuentra en una posición más grande que el número real de elementos de la colección	ORA-6533
SUBSCRIPT_OUTSIDE_LIMIT	El programa está referenciando un elemento de un arreglo utilizando un número fuera del rango permitido (por ejemplo, el elemento “-1”)	ORA-6532
SYS_INVALID_ROWID	La conversión de una cadena de caracteres hacia un tipo rowid falló porque la cadena no representa un número	ORA-1410
TIMEOUT_ON_RESOURCE	Se excedió el tiempo máximo de espera por un recurso en Oracle	ORA-51
TOO_MANY_ROWS	Una sentencia SELECT INTO devuelve más de una fila	ORA-1422
VALUE_ERROR	Ocurrió un error aritmético, de conversión o truncamiento. Por ejemplo, sucede cuando se intenta calzar un valor muy grande dentro de una variable más pequeña	ORA-6502
ZERO_DIVIDE	El programa intentó efectuar una división por cero	ORA-1476

Ejemplo: Hagamos un procedimiento almacenado que reciba dos números como parámetros y muestre su división por pantalla:

```
CREATE OR REPLACE PROCEDURE DIV_NUMEROS(p_n1 IN NUMBER, p_n2 IN NUMBER)
IS
    v_division NUMBER;
BEGIN
    v_division := p_n1 / p_n2;
    DBMS_OUTPUT.PUT_LINE('Resultado: ' || v_division);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Imposible dividir por cero');
END;
/
```

Tenemos a nuestra disposición la variable SQLCODE, que contiene el código de error Oracle para excepciones interna, cuyos valores son:

- 0: No se encontró ninguna excepción.
- 1: Excepción definida por el usuario.
- +100: Excepción NO_DATA_FOUND.
- Número negativo: Código de error que generó la excepción.

También disponemos de la variable SQLERRM, con el mensaje de error asociado.

```
DECLARE
    v_salary empleado.salemp%TYPE;
    e_emp_no_valido EXCEPTION;
    v_codemp empleado.codemp%TYPE := &COD_EMP;
BEGIN
    SELECT salemp INTO v_salary
    FROM empleado
    WHERE codemp = v_codemp;
    DBMS_OUTPUT.PUT_LINE ('Salario: ' || v_salary);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error inesperado:' || SQLCODE || ' - ' ||
SQLERRM);
END;
/
```

3.EXCEPCIONES DEFINIDAS POR EL USUARIO

Para poder utilizar o crear una **excepción definida a nivel a usuario**, lo primero que se debe hacer es declararla en la zona de declaraciones. Las excepciones predefinidas se lanzarán automáticamente asociadas a un error de Oracle. Las excepciones definidas por el usuario deberán definirse y lanzarse explícitamente.

En PL/SQL nosotros podemos **definir nuestras propias excepciones en la sección DECLARE** de cualquier bloque. Estas excepciones podemos **lanzarlas explícitamente** por medio de la sentencia **RAISE nombre_excepción**. Sintaxis:

```
DECLARE
    nombre_excepcion EXCEPTION;
BEGIN
    ...
    RAISE nombre_excepcion;
    ...
EXCEPTION
    WHEN nombre_excepcion THEN
        tratamiento;
END;
```

Ejemplo:

Hagamos un procedimiento anónimo que lea un ID de un empleado por teclado y busque su salario. Lanzará una excepción si el identificador es menor que 1.

```
DECLARE
    v_salary empleado.salemp%TYPE;
    e_emp_no_valido EXCEPTION;
    v_codemp empleado.codemp%TYPE := &COD_EMP;
BEGIN
    IF v_codemp <= 0 THEN
        RAISE e_emp_no_valido;
    END IF;

    SELECT salemp INTO v_salary
    FROM empleado
    WHERE codemp = v_codemp;
    DBMS_OUTPUT.PUT_LINE ('Salario: ' || v_salary);
EXCEPTION
    WHEN e_emp_no_valido THEN
        DBMS_OUTPUT.PUT_LINE('Identificador no válido');
```

```

WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Código no encontrado');
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error inesperado:' || SQLCODE || ' - ' ||
SQLERRM);
END;
/

```

4. MENSAJES DE ERROR DE USUARIO

Oracle también permite que nosotros lancemos **nuestros propios mensajes de error** a las aplicaciones y asociarlos a un código de error que Oracle reserva, para no interferir con los demás códigos de error. Lo hacemos por medio del procedimiento:

```
RAISE_APPLICATION_ERROR(error_number, message);
```

Donde `error_number` es un entero negativo comprendido entre -20000..-20999 y `message` es una cadena que devolvemos a la aplicación.

Ejemplo:

```

CREATE OR REPLACE PROCEDURE SUBIR_SALARIO(p_cod_emple
empleado.codemp%TYPE, p_subida NUMBER(4,2)) IS
    v_salario_actual empleado.salemp%TYPE;
BEGIN
    SELECT salemp INTO v_salario_actual
    FROM empleado
    WHERE codemp = p_cod_emple;

    IF v_salario_actual IS NULL THEN
        RAISE_APPLICATION_ERROR(-20010, 'Salario nulo');
    ELSE
        UPDATE empleado
        SET salemp = salemp * (1 + p_subida / 100)
        WHERE codemp = p_cod_emple;
    END IF;
END;
/

```

A veces, el servidor de Oracle lanza errores que tienen un código (como el ORA-02292 de violación de integridad referencial) pero no tienen un nombre "amigable" predefinido como NO_DATA_FOUND. Esta instrucción permite asociar un nombre inventado por ti a un código de error real de Oracle para poder capturarlo por su nombre.

```

DECLARE
    e_hijos_existentes EXCEPTION;
    -- Asociamos el nombre al código ORA-02292 (borrado de padre con
hijos)
    PRAGMA EXCEPTION_INIT(e_hijos_existentes, -2292);
BEGIN
    DELETE FROM departamento WHERE coddep = 'VENTAS';
EXCEPTION
    WHEN e_hijos_existentes THEN
        DBMS_OUTPUT.PUT_LINE('No puedes borrar el departamento porque
tiene empleados');
END;

```

5. PROPAGACIÓN DE EXCEPCIONES

Este concepto explica qué ocurre cuando una excepción se activa en un bloque que no tiene un manejador específico para ella. En lugar de detener el programa de inmediato, la excepción "viaja" hacia afuera buscando quién la gestione.

Reglas de propagación:

1. **Búsqueda local:** Cuando ocurre un error, PL/SQL busca un WHEN que capture esa excepción en el bloque actual.
2. **Escalada al bloque padre:** Si no lo encuentra, el bloque actual termina bruscamente y la excepción se envía al bloque que lo contiene (bloque padre).
3. **Efecto dominó:** Este proceso se repite sucesivamente hasta encontrar un manejador o llegar al entorno del usuario (como SQL*Plus o una aplicación Java), donde el programa finalmente fallará mostrando el error.

Ejemplo: En este ejemplo, el bloque interno provoca un error que no sabe gestionar, por lo que el bloque externo debe hacerse cargo.

```

DECLARE
    v_test NUMBER;
BEGIN -- Bloque externo
    BEGIN -- Bloque interno
        v_test := 10 / 0; -- Provoca ZERO_DIVIDE
    EXCEPTION
        WHEN NO_DATA_FOUND THEN -- No captura el error de división
            DBMS_OUTPUT.PUT_LINE('No encontrado');
    END;
    -- La excepción ZERO_DIVIDE se propaga aquí al no ser capturada
arriba
EXCEPTION
    WHEN ZERO_DIVIDE THEN -- El bloque externo captura la propagación

```

```
        DBMS_OUTPUT.PUT_LINE('Error capturado en el bloque externo');  
END;  
/
```